DSA STUDY HANDBOOK

ADVANCED DSA — Complete Java Handbook

18 Advanced Topics | Concepts | Algorithms | Java Code • Hinglish explanations, dry runs, complexity analysis, and edge cases.

Contents

#	Topic	Core Paradigm
1	Strongly Connected Components (SCC)	Graph Connectivity & Kosaraju's Two-Pass DFS
2	Dynamic Programming on DAG	Topological Order State Transitions
3	DP on Trees	Postorder Subtree Aggregation
4	Rerooting DP	In-Out / Two-DFS All-Roots Transition
5	Bitmask DP	Subset Compression & State Transitions
6	Digit DP	Bounded Prefix Construction & Tight States
7	Probability DP	Weighted Transition Probabilities
8	Meet in the Middle	Half-Space Splitting & Binary Search
9	Divide and Conquer	Recursive Subproblem Splitting & Merging
10	Coordinate Compression	Rank Mapping for Sparse Large Domains
11	Sweep Line	Event Sorting & Active Interval Tracking
12	Offline Queries	Query Reordering & Incremental Answering
13	Two Heaps	Dual Priority Queues for Dynamic Median
14	Fast and Slow Pointers	Floyd's Cycle Detection & Midpoint Traversal
15	Topological Ordering	Kahn's Indegree BFS & Dependency Resolution
16	Binary Search on Answer	Monotonic Feasibility Predicate Search
17	Prefix and Suffix Preprocessing	O(1) Range Queries & Bidirectional Splits
18	Small-to-Large Merging	DSU on Tree / Heuristic Set Merging

1. Strongly Connected Components (SCC)

Concept

Directed graph ka SCC ek maximal set of vertices hota hai jahan har vertex se doosre har vertex tak path exist karta hai.

Intuition

Agar A se B aur B se A dono reachable hain, toh dono same SCC mein hain. Kosaraju algorithm do DFS passes use karta hai.

Algorithm / Steps

- Original graph par DFS chalao aur finishing order store karo.
- Graph ki saari edges reverse karke transpose graph banao.
- Finishing order ke reverse mein transpose graph par DFS chalao.
- Har DFS traversal ek SCC identify karega.

Java Implementation

```
import java.util.*;

class Kosaraju {
    static void dfs1(int u, List<Integer>[] g,
                     boolean[] vis, Stack<Integer> st) {
        vis[u] = true;
        for (int v : g[u])
            if (!vis[v]) dfs1(v, g, vis, st);
        st.push(u);
    }

    static void dfs2(int u, List<Integer>[] rg,
                     boolean[] vis, List<Integer> comp) {
        vis[u] = true;
        comp.add(u);
        for (int v : rg[u])
            if (!vis[v]) dfs2(v, rg, vis, comp);
    }

    static List<List<Integer>> kosaraju(
            List<Integer>[] g, List<Integer>[] rg) {
        int n = g.length;
        boolean[] vis = new boolean[n];
        Stack<Integer> st = new Stack<>();

        for (int i = 0; i < n; i++)
            if (!vis[i]) dfs1(i, g, vis, st);

        Arrays.fill(vis, false);
        List<List<Integer>> ans = new ArrayList<>();

        while (!st.isEmpty()) {
            int u = st.pop();
            if (!vis[u]) {
                List<Integer> comp = new ArrayList<>();
                dfs2(u, rg, vis, comp);
                ans.add(comp);
            }
        }
        return ans;
    }
}
```

Aspect	Details
Complexity	Time: O(V + E). Space: O(V + E), including graph storage and DFS stack.
Dry Run	Suppose edges are 0→1, 1→2, 2→0 and 2→3. First SCC is {0,1,2}; second SCC is {3}.

Important Edge Cases

- Disconnected directed graph.
- Single vertex without edges.
- Self-loop.
- Multiple SCCs connected in one direction.

Practice Problems

- Find SCCs using Kosaraju.
- Find SCCs using Tarjan.
- Build the condensation DAG.

2. Dynamic Programming on DAG

Concept

DAG par DP tab possible hoti hai kyunki directed cycles nahi hote. Topological order mein process karne par dependencies pehle calculate ho jaati hain.

Intuition

Har node ka answer uske predecessors ya successors ke answers se calculate hota hai.

Algorithm / Steps

- DAG ka topological order nikalo.
- DP array initialize karo.
- Topological order mein nodes process karo.
- Transitions ke according neighbouring states update karo.

Java Implementation

```
import java.util.*;

class DAGDP {
    static int[] longestPath(List<Integer>[] g) {
        int n = g.length;
        int[] indeg = new int[n];
        int[] dp = new int[n];

        for (int u = 0; u < n; u++)
            for (int v : g[u])
                indeg[v]++;

        Queue<Integer> q = new ArrayDeque<>();
        for (int i = 0; i < n; i++)
            if (indeg[i] == 0) q.add(i);

        while (!q.isEmpty()) {
            int u = q.poll();

            for (int v : g[u]) {
                dp[v] = Math.max(dp[v], dp[u] + 1);

                if (--indeg[v] == 0)
                    q.add(v);
            }
        }
        return dp;
    }
}
```

Aspect	Details
Complexity	Time: O(V + E). Space: O(V).
Dry Run	For edges 0→1, 0→2, 1→3, 2→3, the longest path ending at node 3 has length 2.

Important Edge Cases

- Multiple source nodes.
- Isolated nodes.
- Negative edge weights in weighted DAGs.
- Empty graph.

Practice Problems

- Longest path in a DAG.
- Count paths from source to destination.
- Minimum cost path in a DAG.

3. DP on Trees

Concept

Tree DP mein har node ka answer uske children ke answers se calculate hota hai. Tree ko root karke DFS postorder use karte hain.

Intuition

Child ka answer pehle calculate hota hai, phir parent apna answer banata hai.

Algorithm / Steps

- Tree ko kisi root par root karo.
- DFS se parent-child relation identify karo.
- Children ke DP states calculate karo.
- Un states ko combine karke current node ka answer nikalo.

Java Implementation

```
import java.util.*;

class TreeDP {
    static int[] subtreeSize;
    static List<Integer>[] g;

    static void dfs(int u, int p) {
        subtreeSize[u] = 1;

        for (int v : g[u]) {
            if (v == p) continue;
            dfs(v, u);
            subtreeSize[u] += subtreeSize[v];
        }
    }

    static int[] solve(List<Integer>[] graph) {
        g = graph;
        int n = g.length;
        subtreeSize = new int[n];
        dfs(0, -1);
        return subtreeSize;
    }
}
```

Aspect	Details
Complexity	Time: O(N). Space: O(N), including recursion stack.
Dry Run	For tree 0-1, 0-2, 1-3, subtree size of node 0 is 4, node 1 is 2, and node 2 is 1.

Important Edge Cases

- Single node tree.
- Chain-shaped tree.
- Star-shaped tree.
- Deep recursion stack for large trees.

Practice Problems

- Calculate subtree sizes.
- Maximum independent set in a tree.
- Tree diameter using DP.

4. Rerooting DP

Concept

Rerooting DP technique mein tree ka answer har possible root ke liye efficiently calculate karte hain.

Intuition

Har node ko root maan kar DFS dobara chalane ke bajay, ek root ka answer calculate karke adjacent root ka answer O(1) ya O(degree) mein update karte hain.

Algorithm / Steps

- Initial root choose karo.
- First DFS se subtree information calculate karo.
- Initial root ka answer calculate karo.
- Second DFS mein parent answer se child answer derive karo.

Java Implementation

```
import java.util.*;

class Rerooting {
    static List<Integer>[] g;
    static long[] sub, ans;

    static void dfs1(int u, int p, int depth) {
        sub[u] = 1;
        ans[0] += depth;

        for (int v : g[u]) {
            if (v == p) continue;
            dfs1(v, u, depth + 1);
            sub[u] += sub[v];
        }
    }

    static void dfs2(int u, int p, int n) {
        for (int v : g[u]) {
            if (v == p) continue;

            ans[v] = ans[u] + n - 2 * sub[v];
            dfs2(v, u, n);
        }
    }

    static long[] solve(List<Integer>[] graph) {
        g = graph;
        int n = g.length;
        sub = new long[n];
        ans = new long[n];

        dfs1(0, -1, 0);
        dfs2(0, -1, n);
        return ans;
    }
}
```

Aspect	Details
Complexity	Time: O(N). Space: O(N). This example calculates sum of distances from every node to all other nodes.
Dry Run	If a child subtree has size 2 in a tree of 5 nodes, moving root across that edge changes the distance sum by 5 - 2*2 = 1.

Important Edge Cases

- Single node.
- Chain tree.
- Large subtree.
- Integer overflow: use long.

Practice Problems

- Sum of distances from every node.
- Maximum distance sum after choosing root.
- Count valid configurations for every root.

5. Bitmask DP

Concept

Bitmask DP mein subsets ko integer masks se represent karte hain. N usually small hota hai, often 15-20.

Intuition

Mask ke har bit se pata chalta hai ki corresponding element selected hai ya nahi.

Algorithm / Steps

- State define karo: dp[mask].
- Initial state set karo.
- Mask ke unset bits par transitions lagao.
- Final mask ya required states ka answer lo.

Java Implementation

```
import java.util.*;

class BitmaskDP {
    static int tsp(int[][] cost) {
        int n = cost.length;
        int total = 1 << n;
        int INF = 1_000_000_000;

        int[][] dp = new int[total][n];
        for (int[] row : dp)
            Arrays.fill(row, INF);

        dp[1][0] = 0;

        for (int mask = 0; mask < total; mask++) {
            for (int u = 0; u < n; u++) {
                if (dp[mask][u] == INF) continue;
                if ((mask & (1 << u)) == 0) continue;

                for (int v = 0; v < n; v++) {
                    if ((mask & (1 << v)) != 0) continue;

                    int next = mask | (1 << v);
                    dp[next][v] = Math.min(
                        dp[next][v],
                        dp[mask][u] + cost[u][v]
                    );
                }
            }
        }

        int full = total - 1;
        int ans = INF;

        for (int u = 1; u < n; u++)
            ans = Math.min(ans,
                dp[full][u] + cost[u][0]);

        return ans;
    }
}
```

Aspect	Details
Complexity	Time: $O(N^2 \times 2^N)$. Space: $O(N \times 2^N)$.
Dry Run	For 3 cities, masks range from 000 to 111. Starting mask 001 means only city 0 is visited.

Important Edge Cases

- N = 1.
- Large N causing memory issues.
- Unreachable transitions.
- Use long if path costs may overflow int.

Practice Problems

- Travelling Salesman Problem.
- Assignment problem using bitmask DP.
- Count Hamiltonian paths.

6. Digit DP

Concept

Digit DP large numeric range ke andar properties satisfy karne wale numbers count karne ke liye use hoti hai.

Intuition

Number ko digit-by-digit build karte hain. State mein position, tight, leading-zero aur required property store hoti hai.

Algorithm / Steps

- Range [L, R] ka answer F(R) - F(L-1) banao.
- Number ko string mein convert karo.
- DFS(pos, tight, started, state) define karo.
- Memoization se repeated states save karo.

Java Implementation

```
import java.util.*;

class DigitDP {
    static String s;
    static long[][][] memo;

    static long dfs(int pos, int tight, int started) {
        if (pos == s.length()) return 1;

        if (memo[pos][tight][started] != -1)
            return memo[pos][tight][started];

        int limit = tight == 1 ? s.charAt(pos) - '0' : 9;
        long ans = 0;

        for (int d = 0; d <= limit; d++) {
            int nt = (tight == 1 && d == limit) ? 1 : 0;
            int ns = (started == 1 || d != 0) ? 1 : 0;

            ans += dfs(pos + 1, nt, ns);
        }

        return memo[pos][tight][started] = ans;
    }

    static long countUpTo(long n) {
        if (n < 0) return 0;

        s = Long.toString(n);
        memo = new long[s.length()][2][2];

        for (long[][] a : memo)
            for (long[] b : a)
                Arrays.fill(b, -1);

        return dfs(0, 1, 0);
    }

    static long countRange(long l, long r) {
        return countUpTo(r) - countUpTo(l - 1);
    }
}
```

Aspect	Details
Complexity	Time: O(D × states × 10), where D is number of digits. Space: O(D × states).
Dry Run	countRange(1, 20) counts 20 integers. More complex digit DP adds states for digit sum, remainder, or previous digit.

Important Edge Cases

- L = 0.
- Leading zero handling.
- Negative lower bound.
- Long overflow in counts.

Practice Problems

- Count numbers with digit sum K.
- Count numbers without consecutive equal digits.
- Count numbers divisible by K.

7. Probability DP

Concept

Probability DP mein states ke saath event ki probability store karte hain. Transition probabilities ka weighted sum hota hai.

Intuition

Har possible previous state se current state tak aane ki probability add karte hain.

Algorithm / Steps

- State define karo.
- Initial probability initialize karo.
- Har possible transition probability calculate karo.
- Required final states ki probability sum karo.

Java Implementation

```
class ProbabilityDP {
    // Probability of getting exactly k heads
    // after n independent fair coin tosses.
    static double solve(int n, int k) {
        double[] dp = new double[k + 1];
        dp[0] = 1.0;

        for (int toss = 1; toss <= n; toss++) {
            for (int j = Math.min(k, toss); j >= 0; j--) {
                double stay = dp[j] * 0.5;
                double come = j > 0 ? dp[j - 1] * 0.5 : 0.0;
                dp[j] = stay + come;
            }
        }
        return dp[k];
    }
}
```

Aspect	Details
Complexity	Time: O(NK). Space: O(K).
Dry Run	For 2 fair tosses, probability of exactly 1 head is 2/4 = 0.5.

Important Edge Cases

- Probability 0 or 1.
- Floating point precision.
- Non-uniform probabilities.
- Impossible states.

Practice Problems

- Probability of at least K heads.
- Expected number of coin tosses.
- Random walk probability.

8. Meet in the Middle

Concept

Meet in the Middle exponential search space ko do halves mein divide karta hai. Usually N around 40 tak useful hota hai.

Intuition

2^N possibilities ke bajay do groups ke $2^{N/2}$ subsets generate karte hain.

Algorithm / Steps

- Array ko two halves mein split karo.
- First half ke subset sums generate karo.
- Second half ke subset sums generate karo.
- Sorting aur binary search se best compatible pair find karo.

Java Implementation

```
import java.util.*;

class MeetInMiddle {
    static List<Long> sums(int[] a, int l, int r) {
        List<Long> res = new ArrayList<>();
        int n = r - l;
        for (int mask = 0; mask < (1 << n); mask++) {
            long sum = 0;
            for (int i = 0; i < n; i++)
                if ((mask & (1 << i)) != 0)
                    sum += a[l + i];
            res.add(sum);
        }
        return res;
    }

    static long best(int[] a, long target) {
        int mid = a.length / 2;
        List<Long> x = sums(a, 0, mid);
        List<Long> y = sums(a, mid, a.length);
        Collections.sort(y);

        long ans = Long.MIN_VALUE;

        for (long v : x) {
            int idx = Collections.binarySearch(y, target - v);
            if (idx < 0) idx = -idx - 2;
            if (idx >= 0) {
                ans = Math.max(ans, v + y.get(idx));
            }
        }
        return ans;
    }
}
```

Aspect	Details
Complexity	Time: $O(N \times 2^{N/2})$. Space: $O(2^{N/2})$.
Dry Run	For a = [3, 34, 4, 12] and target = 16, left sums = [0, 3, 34, 37] and right sums = [0, 4, 12, 16]. Pair (0, 16) gives 16.

Important Edge Cases

- N = 0 or N = 1 (odd lengths).
- Negative numbers in the input array.
- Target smaller than all non-empty subset sums.
- Duplicate subset sums when counting exact matches.

Practice Problems

- Maximum subset sum ≤ target.
- Count subsets with exact sum S (N ≤ 40).
- 4-Sum II (four arrays tuple sum).

9. Divide and Conquer

Concept

Divide and Conquer problem ko independent subproblems mein split karta hai, recursively solve karta hai, aur results ko combine karta hai.

Intuition

Large problem ko halves mein todne se recursion depth O(log N) rehti hai, jisse $O(N^2)$ operations O(N log N) mein reduce ho jaate hain.

Algorithm / Steps

- Base case define karo jab subarray size 1 ya small ho.
- Midpoint calculate karke problem ko left aur right halves mein divide karo.
- Dono halves ko recursively solve karo.
- Cross-boundary pairs ya results ko linear time mein merge karo.

Java Implementation

```
class InversionCount {
    static long mergeSort(int[] a, int[] temp, int l, int r) {
        if (l >= r) return 0;
        int mid = l + (r - l) / 2;
        long inv = mergeSort(a, temp, l, mid) + mergeSort(a, temp, mid + 1, r);

        int i = l, j = mid + 1, k = l;
        while (i <= mid && j <= r) {
            if (a[i] <= a[j]) temp[k++] = a[i++];
            else {
                temp[k++] = a[j++];
                inv += (mid - i + 1);
            }
        }
        while (i <= mid) temp[k++] = a[i++];
        while (j <= r) temp[k++] = a[j++];
        for (i = l; i <= r; i++) a[i] = temp[i];
        return inv;
    }
}
```

Aspect	Details
Complexity	Time: O(N log N). Space: O(N).
Dry Run	For [2, 4, 1, 3], left [2, 4] and right [1, 3] merge: 1 is smaller than 2 and 4 (+2), 3 is smaller than 4 (+1). Total = 3.

Important Edge Cases

- Already sorted or reverse-sorted arrays.
- Duplicate elements (use ≤ for strict inversions).
- Inversion count exceeding 32-bit int (use long).
- Empty or single-element array.

Practice Problems

- Count inversions in an array.
- Reverse pairs (a[i] > 2 × a[j]).
- Closest pair of points in 2D plane.

10. Coordinate Compression

Concept

Coordinate Compression large range ke sparse values (e.g., -10^9 se 10^9) ko compact ranks [0, K−1] mein map karta hai, relative order preserve karte hue.

Intuition

Jab sirf values ki relative ordering matter karti hai (jaise Fenwick Tree ya Segment Tree indexing mein), toh actual magnitude ki zaroorat nahi hoti.

Algorithm / Steps

- Saare relevant coordinates ko ek temporary array mein collect karo.
- Array ko sort karo aur duplicates remove karo.
- Har original value ko binary search se uska 0-based index assign karo.
- Compressed indices par Fenwick Tree ya Segment Tree run karo.

Java Implementation

```
import java.util.*;

class CoordinateCompression {
    static int[] compress(int[] a) {
        int[] sorted = a.clone();
        Arrays.sort(sorted);

        int m = 0;
        for (int x : sorted) {
            if (m == 0 || sorted[m - 1] != x)
                sorted[m++] = x;
        }

        int[] rank = new int[a.length];
        for (int i = 0; i < a.length; i++) {
            rank[i] = Arrays.binarySearch(sorted, 0, m, a[i]);
        }
        return rank;
    }
}
```

Aspect	Details
Complexity	Time: O(N log N). Space: O(N).
Dry Run	Input [500, −10, 500, 10000] sorts to [−10, 500, 10000]. Output ranks become [1, 0, 1, 2].

Important Edge Cases

- All elements identical (compresses to all 0s).
- Negative numbers and Integer.MIN_VALUE / MAX_VALUE.
- Forgetting to include query endpoints during offline compression.
- 1-based indexing requirement for Fenwick Trees (add +1 to rank).

Practice Problems

- Count of smaller numbers after self.
- 2D rectangle union area.
- Dynamic range frequency queries on large values.

11. Sweep Line

Concept

Sweep Line ek imaginary vertical line ko sorted events ke across move karke geometric ya interval problems solve karta hai.

Intuition

Continuous 2D ya interval space ko discrete event points (start aur end) mein convert karke active set maintain karte hain.

Algorithm / Steps

- Har interval ya segment ko start (+1) aur end (−1) events mein break karo.
- Events ko coordinate ke according sort karo (tie-breaking carefully define karo).
- Sorted events par iterate karte hue active counter ya balanced BST update karo.
- Har step par optimal answer record karo.

Java Implementation

```
import java.util.*;

class SweepLine {
    static int maxOverlappingIntervals(int[][] intervals) {
        List<int[]> events = new ArrayList<>();
        for (int[] in : intervals) {
            events.add(new int[]{in[0], 1});
            events.add(new int[]{in[1], -1});
        }
        // End (-1) before Start (+1) at same point for half-open [l, r)
        events.sort((a, b) -> a[0] != b[0] ? Integer.compare(a[0], b[0])
                                           : Integer.compare(a[1], b[1]));

        int active = 0, maxActive = 0;
        for (int[] e : events) {
            active += e[1];
            maxActive = Math.max(maxActive, active);
        }
        return maxActive;
    }
}
```

Aspect	Details
Complexity	Time: O(N log N). Space: O(N).
Dry Run	For [1, 5], [2, 6], [5, 8]: events at 1(+1), 2(+1), 5(−1), 5(+1), 6(−1), 8(−1). Max active = 2.

Important Edge Cases

- Closed [l, r] vs half-open [l, r) tie-breaking rules.
- Zero-length intervals [x, x].
- Duplicate intervals.
- Large coordinates requiring long instead of int.

Practice Problems

- Minimum meeting rooms required.
- The Skyline Problem.
- Rectangle area union.

12. Offline Queries

Concept

Offline Queries mein saari queries pehle read karke unhe favorable order mein sort karte hain, taaki data structure ko ek hi pass mein update karke answer kiya ja sake.

Intuition

Online order mein baar-baar structure reset karne ke bajay, right-endpoint ya threshold ke according sort karke incremental updates use karte hain.

Algorithm / Steps

- Har query ke saath uska original index store karo.
- Queries ko right boundary R ya value threshold K ke basis par sort karo.
- Array pointers ko monotonic way mein advance karo aur BIT/DSU update karo.
- Answers ko unke original query index par store karo.

Java Implementation

```
import java.util.*;

class OfflineQueries {
    // Count elements <= k in prefix [0..r] for each query {r, k, idx}
    static int[] solve(int[] a, int[][] queries) {
        int n = a.length, q = queries.length;
        int[][] items = new int[n][2];
        for (int i = 0; i < n; i++) items[i] = new int[]{a[i], i + 1};
        Arrays.sort(items, Comparator.comparingInt(x -> x[0]));

        Arrays.sort(queries, Comparator.comparingInt(x -> x[1]));
        int[] bit = new int[n + 1];
        int[] ans = new int[q];
        int ptr = 0;

        for (int[] qry : queries) {
            int r = qry[0], k = qry[1], id = qry[2];
            while (ptr < n && items[ptr][0] <= k) {
                for (int i = items[ptr][1]; i <= n; i += i & -i) bit[i]++;
                ptr++;
            }
            int sum = 0;
            for (int i = r + 1; i > 0; i -= i & -i) sum += bit[i];
            ans[id] = sum;
        }
        return ans;
    }
}
```

Aspect	Details
Complexity	Time: O((N + Q) log N). Space: O(N + Q).
Dry Run	Sorting both elements and queries by value K ensures each element is inserted into the Fenwick Tree at most once.

Important Edge Cases

- Forgetting to restore original query order in output array.
- Queries with empty ranges (L > R).
- Dynamic mutations interleaved with queries (requires Mo's with updates or persistent DS).
- Duplicate query thresholds.

Practice Problems

- Number of distinct elements in subarray [L, R] (DQUERY).
- Mo's Algorithm range sum / mode queries.
- Minimum spanning tree edge weight threshold queries.

13. Two Heaps

Concept

Two Heaps pattern stream of numbers ko do halves mein partition karta hai: lower half ke liye Max-Heap aur upper half ke liye Min-Heap.

Intuition

Dono heaps ke tops par middle elements rehte hain, jisse O(1) mein median milta hai aur O(log N) mein insertion hota hai.

Algorithm / Steps

- Lower half ke liye Max-Heap aur upper half ke liye Min-Heap banao.
- Naya element pehle Max-Heap mein daalo, phir uska top Min-Heap mein shift karo.
- Size invariant maintain karo: maxHeap.size() ≥ minHeap.size() aur difference ≤ 1.
- Median ke liye maxHeap.peek() ya dono tops ka average return karo.

Java Implementation

```
import java.util.*;

class MedianFinder {
    PriorityQueue<Integer> low = new PriorityQueue<>(Collections.reverseOrder());
    PriorityQueue<Integer> high = new PriorityQueue<>();

    void addNum(int num) {
        low.offer(num);
        high.offer(low.poll());
        if (low.size() < high.size()) {
            low.offer(high.poll());
        }
    }

    double findMedian() {
        if (low.size() > high.size()) return low.peek();
        return (low.peek() + high.peek()) / 2.0;
    }
}
```

Aspect	Details
Complexity	Time: O(log N) per insertion, O(1) per median query. Space: O(N).
Dry Run	Insert 5, 2, 10: low=[5, 2], high=[10]. Since low has size 2 and high has size 1, median is low.peek() = 5.

Important Edge Cases

- Integer overflow during (low.peek() + high.peek()) / 2.0 (cast to double/long first).
- Querying median on empty stream.
- Sliding window median requiring lazy deletion map.
- Duplicate numbers across the split boundary.

Practice Problems

- Find Median from Data Stream.
- Sliding Window Median.
- IPO (Maximize Capital using two heaps).

14. Fast and Slow Pointers

Concept

Fast and Slow Pointers (Tortoise and Hare) do pointers ko alag speeds (1 step vs 2 steps) par move karke cycle detection aur midpoint finding karta hai.

Intuition

Agar linked list ya state graph mein cycle hai, toh fast pointer har step par relative distance 1 se kam karke slow pointer se zaroor milega.

Algorithm / Steps

- Slow aur fast dono pointers ko head par initialize karo.
- Loop chalao jab tak fast aur fast.next null na hon: slow = slow.next, fast = fast.next.next.
- Agar slow == fast ho jaaye, toh cycle exist karti hai.
- Cycle start nikalne ke liye ek pointer ko head par reset karo aur dono ko 1-1 step chalao.

Java Implementation

```
class FastSlowPointers {
    static class Node {
        int val;
        Node next;
        Node(int v) { val = v; }
    }

    static Node detectCycleStart(Node head) {
        Node slow = head, fast = head;
        while (fast != null && fast.next != null) {
            slow = slow.next;
            fast = fast.next.next;
            if (slow == fast) {
                Node entry = head;
                while (entry != slow) {
                    entry = entry.next;
                    slow = slow.next;
                }
                return entry;
            }
        }
        return null;
    }
}
```

Aspect	Details
Complexity	Time: O(N). Space: O(1).
Dry Run	List 1→2→3→4→2: pointers meet at node 4. Resetting entry to 1 and moving both by 1 meets at cycle start node 2.

Important Edge Cases

- Empty list (head == null) or single node without loop.
- Self-loop on head node.
- Even vs odd length list when finding middle node.
- NullPointerException if fast.next is not checked before fast.next.next.

Practice Problems

- Linked List Cycle II (Find start of cycle).
- Find the Duplicate Number in O(1) space.
- Happy Number cycle detection.

15. Topological Ordering

Concept

Topological Ordering ek Directed Acyclic Graph (DAG) ke vertices ka linear ordering hai jahan har directed edge u→v ke liye u, v se pehle aata hai.

Intuition

Jis node ki indegree 0 hai uski saari prerequisites complete ho chuki hain, isliye usse schedule karke uske neighbours ki indegree kam kar sakte hain.

Algorithm / Steps

- Har vertex ki indegree count karo.
- Indegree 0 wale saare vertices ko queue mein push karo.
- Queue se node pop karo, result mein add karo, aur uske neighbours ki indegree decrement karo.
- Agar processed nodes count < N hai, toh graph mein cycle exist karti hai.

Java Implementation

```
import java.util.*;

class TopologicalSort {
    static List<Integer> kahn(int n, List<Integer>[] g) {
        int[] indeg = new int[n];
        for (int u = 0; u < n; u++)
            for (int v : g[u]) indeg[v]++;

        Queue<Integer> q = new ArrayDeque<>();
        for (int i = 0; i < n; i++)
            if (indeg[i] == 0) q.add(i);

        List<Integer> order = new ArrayList<>();
        while (!q.isEmpty()) {
            int u = q.poll();
            order.add(u);
            for (int v : g[u]) {
                if (--indeg[v] == 0) q.add(v);
            }
        }
        return order.size() == n ? order : Collections.emptyList();
    }
}
```

Aspect	Details
Complexity	Time: O(V + E). Space: O(V).
Dry Run	Edges 0→1, 0→2, 1→3, 2→3: Queue starts with [0], pops 0, enqueues 1 and 2, then pops 1, 2, 3.

Important Edge Cases

- Graph containing a directed cycle (returns empty list).
- Multiple valid topological orderings (use PriorityQueue for lexicographically smallest).
- Disconnected components with zero edges.
- Parallel duplicate edges inflating indegree.

Practice Problems

- Course Schedule I & II.
- Alien Dictionary character ordering.
- Find Eventual Safe States.

16. Binary Search on Answer

Concept

Jab optimal answer directly find karna mushkil ho, lekin kisi candidate value X ke liye feasibility check O(N) mein ho sake aur predicate monotonic ho, tab Answer Space par Binary Search lagate hain.

Intuition

Agar capacity X par task possible hai, toh har Y > X par bhi possible hoga; is monotonicity se minimum valid X O(log(Range)) checks mein mil jaata hai.

Algorithm / Steps

- Search space ke tight bounds [low, high] define karo.
- Monotonic boolean function canAchieve(mid) likho.
- Agar canAchieve(mid) true hai, toh ans = mid save karke left half (high = mid − 1) search karo.
- Warna right half (low = mid + 1) mein search karo.

Java Implementation

```
class BinarySearchAnswer {
    static boolean canShip(int[] weights, int days, long cap) {
        int needed = 1;
        long current = 0;
        for (int w : weights) {
            if (current + w > cap) {
                needed++;
                current = 0;
            }
            current += w;
        }
        return needed <= days;
    }

    static long shipWithinDays(int[] weights, int days) {
        long low = 0, high = 0;
        for (int w : weights) {
            low = Math.max(low, w);
            high += w;
        }
        long ans = high;
        while (low <= high) {
            long mid = low + (high - low) / 2;
            if (canShip(weights, days, mid)) {
                ans = mid;
                high = mid - 1;
            } else {
                low = mid + 1;
            }
        }
        return ans;
    }
}
```

Aspect	Details
Complexity	Time: O(N log(Sum − Max)). Space: O(1).
Dry Run	Weights [1,2,3,4,5], days = 2: low=5, high=15. mid=10 works (1+2+3+4 | 5), mid=8 fails, mid=9 works (1+2+3 | 4+5). Ans = 9.

Important Edge Cases

- Setting low too small (must be at least max(weights)).
- Integer overflow in low + high (always use low + (high − low) / 2 and long).
- Floating-point binary search requiring fixed iterations (e.g., 100 iterations).
- Non-monotonic predicate where binary search is invalid.

Practice Problems

- Aggressive Cows / Magnetic Force Between Two Balls.
- Allocate Minimum Number of Pages.
- Koko Eating Bananas.

17. Prefix and Suffix Preprocessing

Concept

Prefix aur Suffix arrays cumulative information (sum, min, max, gcd, product) ko precompute karte hain taaki range queries ya bidirectional splits O(1) mein answer ho sakein.

Intuition

Index i ke left aur right parts ka combined result bina nested loops ke O(1) mein mil jaata hai.

Algorithm / Steps

- Left-to-right pass mein pref[i] compute karo.
- Right-to-left pass mein suff[i] compute karo.
- Har index i par pref[i − 1] aur suff[i + 1] ko combine karke answer nikalo.
- Boundary indices (i = 0 aur i = N − 1) ke liye identity element use karo.

Java Implementation

```
class PrefixSuffix {
    static long[] productExceptSelf(int[] a) {
        int n = a.length;
        long[] res = new long[n];

        long pref = 1;
        for (int i = 0; i < n; i++) {
            res[i] = pref;
            pref *= a[i];
        }

        long suff = 1;
        for (int i = n - 1; i >= 0; i--) {
            res[i] *= suff;
            suff *= a[i];
        }
        return res;
    }
}
```

Aspect	Details
Complexity	Time: O(N). Space: O(1) auxiliary space (excluding output array).
Dry Run	For [1, 2, 3, 4]: prefix pass gives [1, 1, 2, 6]; suffix pass multiplies by [24, 12, 4, 1] → [24, 12, 8, 6].

Important Edge Cases

- Zeros in the input array (handled cleanly without division).
- Single element array (returns identity [1]).
- 2D prefix sum inclusion-exclusion boundary indices.
- Overflow in cumulative products or sums (use long).

Practice Problems

- Product of Array Except Self.
- Trapping Rain Water (prefix max & suffix max).
- Maximum GCD of array after removing one element.

18. Small-to-Large Merging

Concept

Small-to-Large Merging (DSU on Tree / Sack) technique mein jab do sets ya maps ko merge karte hain, toh hamesha chhote set ke elements ko bade set mein insert karte hain.

Intuition

Jab bhi koi element move hota hai, uske naye set ka size kam se kam double ho jaata hai; isliye ek element poore execution mein maximum O(log N) baar move hota hai.

Algorithm / Steps

- Har node ke subtree ke liye ek Set ya HashMap maintain karo.
- DFS mein har child subtree ko recursively solve karo.
- Agar child set ka size current parent set se bada hai, toh dono references swap kar do.
- Chhote set ke saare elements bade set mein add karo.

Java Implementation

```
import java.util.*;

class SmallToLarge {
    static List<Integer>[] g;
    static int[] color, distinctCount;

    static Set<Integer> dfs(int u, int p) {
        Set<Integer> curr = new HashSet<>();
        curr.add(color[u]);

        for (int v : g[u]) {
            if (v == p) continue;
            Set<Integer> child = dfs(v, u);
            if (child.size() > curr.size()) {
                Set<Integer> temp = curr;
                curr = child;
                child = temp;
            }
            curr.addAll(child);
        }
        distinctCount[u] = curr.size();
        return curr;
    }
}
```

Aspect	Details
Complexity	Time: $O(N \log^2 N)$ with TreeSet/Map, $O(N \log N)$ expected with HashSet. Space: $O(N)$.
Dry Run	If parent set has 1 element and child set has 50 elements, swapping references takes $O(1)$ and only 1 element is copied.

Important Edge Cases

- Deeply skewed chain trees ($O(N^2)$ without small-to-large swap).
- Copying sets by value instead of swapping object references.
- Maintaining auxiliary aggregates (like max frequency or mode) during merge.
- Deep recursion stack on $N = 2 \times 10^5$ trees.

Practice Problems

- Distinct Colors in Each Subtree (CSES).
- Tree Requests (Subtree character frequency palindrome check).
- Lomsat gelral (Dominant colors sum in subtrees).